二叉树结点定义

```
struct Node{
    int val;
    Node* left;
    Node* right;
    Node(int v):val(v),left(NULL),right(NULL){}
}
```

前序遍历

```
void preorder(Node* root){
    if(!root) return;

    cout<<root->val<<" ";
    preorder(root->left);
    preorder(root->right);
}
```

中序遍历

```
void inorder(Node* root){
    if(!root) return;

    inorder(root->left);
    cout<<root->val<<" ";
    inorder(root->right);
}
```

后序遍历

```
void postorder(Node* root){
    if(!root) return;

    postorder(root->left);
    postorder(root->right);
    cout<<root->val<<" ";
}
```

层序遍历

```
#include<queue>

void levelorder(Node* root){
    if(!root) return;

    queue<Node*> q;
    q.push(root);
```

```
    while(!q.empty()){
        Node* cur = q.front();
        q.pop();

        cout<<cur->val<<" ";

        if(cur->left) q.push(cur->left);
        if(cur->right) q.push(cur->right);
    }
}
```

Kruskal求无向网的最小生成树

输入：

输入数据第一行为两个正整数n（1<n<=30）和m（1<m<100），分别表示顶点数和边数。后面紧跟m行数据，每行数据是一条边的信息，包括三个数字，分别表示该边的两个顶点和边上的权值。

输出：

按顺序输出Kruskal算法求得的最小生成树的边集，每行一条边，包括三个数字，分别是该边的两个顶点和边上的权值，其中第一个顶点的编号应小于第二个顶点的编号。

示例输入

8 11

1 2 3

1 4 5

1 6 18

2 4 7

2 5 6

3 5 10

3 8 20

4 6 15

4 7 11

5 7 8

5 8 12

示例输出

1 2 3

1 4 5

2 5 6

5 7 8

3 5 10

5 8 12

4 6 15

```
#include<bits/stdc++.h>
using namespace std;
struct Edge{
    int u,v,w;
};
Edge e[100];
int parent[31];
int cmp(const void *a,const void *b){
    return ((Edge*)a)->w-((Edge*)b)->w;
}
int find(int x){
    while(parent[x]!=x) x = parent[x];
    return x;
}
int main(){
    int n,m;
    cin >> n >> m;
    for(int i=0;i<m;i++){
        cin >>e[i].u>>e[u].v>>e[i].w;
        if(e[i].u>e[i].v) swap(e[i].u,e[i].v);
    }

    qsort(e,m,sizeof(Edge),cmp);
    for(int i=1;i<=n;i++) parent[i] = i;

    int cnt = 0;
    //最小生成树的停止条件“不是顶点遍历完”，而是选够“n-1”条
    for(int i=0;cnt<n-1;i++){
        if(find(e[i].u)!=find(e[i].v)){
            parent[find(e[i].u)] = find(e[i].v);
            cout<<e[i].u<<" "<<e[i].v<<" "<<e[i].w<<endl;
            cnt++;
        }
    }
    return 0;
}
```

用除留余数法和线性探测法再散列的冲突解决方法构造哈希表

输入：

　输入数据第一行为两个正整数分别为：哈希表表长m（m<100）和除数p（p<=m）。后面每一行是一个整数关键字，以-1作为输入的结束。

输出：

若输入的关键字在哈希表中已存在，则输出该关键字在哈希表中的位置，继续等待输入下一个关键字。

若输入的关键字在哈希表中不存在，则判断当前哈希表中关键字的个数是否等于m-1，若相等，则输出“Table full”，程序结束；否则将关键字插入哈希表，并输出该关键字插入在哈希表中的位置，继续等待输入下一个关键字。

示例输入：

5 3

1 2 3 4 5 -1

示例输出：

1

2

0

3

Table full

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int m,p;
    cin >> m >> p;
    int table[100];
    for(int i = 0;i<m;i++){
        table[i] = -1;
    }

    int cnt = 0;
    int key;

    while(cin>>key&&key!=-1){
        int h = key%p;
        int pos = h;

        while(table[pos]!=-1&&table[pos]!=key){
            pos = (pos+1)%m;
        }

        if(table[pos]==key){
            cout<<pos<<endl;
            continue;
        }

        if(cnt==m-1){
            cout<<"Table full"<<endl;
            break;
        }

        table[pos] = key;
        cnt++;
        cout<<pos<<endl;
    }
    return 0;
}
```

快速排序算法隐含递归树后序遍历

描述：

快速排序递归算法隐含一棵由关键字生成的二叉树（递归树），输出该隐含二叉树的后序遍历序列。（注：划分时以第一关键字为枢轴）

输入说明：

输入数据第一行为1个正整数n，表示关键字个数。第2行为n个整数表示n个关键字。

输出说明：

在一行上输出由关键字隐含的二叉树的后序遍历序列。

输入样例：

9

49 38 65 97 13 27 49 55 4

输出样例：

27 13 38 4 49 55 65 97 49

提示

在快速排序的过程中输出隐含二叉树的后序遍历序列，不用生成二叉树。

```
#include<bits/stdc++.h>
using namespace std;
void quickPostOrder(vector<int>&a,int l,int r){
    if(l>r) return;

    int pivot = a[l];
    int i = l,j = r;

    while(i<j){
        while(i<j&&a[j]>=pivot) j--;
        if(i<j) a[i++] = a[j];
        while(i<j&&a[i]<=pivot) i++;//注意两边都要有等于
        if(i<j) a[j--] = a[i];
    }

    a[i] = pivot;//枢轴归为

    quickPostOrder(a,l,i-1);
    quickPostOrder(a,i+1,r);

    //后序遍历 最后输出根
    static bool first = true;
    if(!first) cout<<" ";
    cout<< pivot;
    first = false;
}
int main(){
    int n;
    cin >> n;
    vector<int> a(n);
    for(int i=0;i<n;i++) cin>>a[i];

    quickPostOrder(a,0,n-1);
    return 0;
}
```

有序表的合并去重

时间限制
1 S
内存限制
10000Kb
问题描述
已知两个有序线性表L1和L2，每个线性表中数据元素的值为单调增的正整数（<100个），各线性表内部无重复元素。把L2中的元素合并到L1中，要求L1中数据元素的值仍为单调递增，且无重复元素。
问题输入
第一行输入两个正整数a,b，分别表示第一个线性表L1和第二个线性表L2的长度；第2行依次输入L1中单调增的a个正整数；第3行依次输入L2中单调增的b个正整数。正整数均小于10000。
问题输出
共有1行，最终合并并去重后的结果，每个数字之间有一个空格。
输入样例
6 7
1 2 5 6 10 13
1 3 5 8 19 22 31
输出样例
1 2 3 5 6 8 10 13 19 22 31

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int a,b;
    cin >> a >> b;
    vector<int> L1(100);
    vector<int> L2(100);

    for(int i=0;i<a;i++) cin>>L1[i];
    for(int i=0;i<b;i++) cin>>L2[i];

    int i=0,j=0;
    bool first = true;

    while(i<a&&j<b){
        int x;
        if(L1[i]<L2[j]){
            x = L1[i++];
        }else if(L1[i]>L2[j]){
            x = L2[j++];
        }else{
            x = L1[i];
            i ++;
            j ++;
        }

        if(!first) cout<<" ";
        cout<<x;
        first = false;
    }

    while(i<a){
        if(!first) cout<<" ";
        cout<<L1[i++];
```

```
        first = false;
    }

    while(j<b){
        if(!first) cout<<" ";
        cout<<L2[j++];
        first = false;
    }

    return 0;
}
```

队列中的元素排序

时间限制：1 秒

内存限制：10KB

问题描述

给定一个队列，请用一系列合法的队列操作函数，比如判断队列是否为空，入队操作，出队操作，将队列中的元素从小到大排序。

输入说明

输入首先给出1个正整数N(N<1000)，表示队列中元素的个数。随后按入队的顺序给出N个整数。

输出说明

在一行中输出排序后出对的序列。数字间以空格分隔，但末尾不得有多余空格。

输入样例

10 3 6 4 8 7 2 9 0 1 5

输出样例

0 1 2 3 4 5 6 7 8 9

提示

可以使用一个辅助的队列，采用循环筛选法(即第一轮将最小的选出来，第二轮将次小的选出来，以此类推)

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int N;
    cin>>N;
    queue<int> q,tmp;
    for(int i=0;i<N;i++){
        int x;
        cin >> x;
        q.push(x);
    }

    bool first = true;
    while(!q.empty){
        int sz = q.size();
        int minVal = q.front();
```

```
        for(int i=0;i<N;i++){
            int x = q.front();q.pop();
            if(x<minVal) minVal = x;
            q.push(x);
        }

        for(int i=0;i<N;i++){
            int x = q.front();q.pop();
            if(x == minVal){
                if(!first) cout<<" ";
                cout<<x;
                first = false;
            }else{
                tmp.push(x);
            }
        }

        q = tmp;
        while(!tmp.empty()) tmp.pop();
    }
    return 0;
}
```

统计二叉树中的叶子结点数

问题描述

建立二叉链表，统计二叉树中的叶子结点数并输出。

输入说明

按照完全二叉树的形式输入二叉树的各结点数据(字符)，其中虚结点用'@'表示。输入以'#'结束。

输出说明

输出叶子结点的个数及具体值。第一行为叶子结点的数据值，各数据用空格分隔，第二行为叶子结点的个数。

输入样例

abc@@de#

输出样例

b d e

3

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    vector<char> a(1000);
    char x;
    int n=1;
    while(cin>>x&&x!='#'){
        a[n++] = x;
    }
```

```
    int cnt = 0;
    bool first = true;
    for(int i=1;i<n;i++){
        if(a[i]=='@') continue;

        bool isleaf = false;

        //因为初始化n++自增 ，所以n比结点数大1 用(n-1)/2没问题
        if(i>(n-1)/2){
            isleaf = true;
        }else if(2*i<n&&2*i+1<n&&a[2*i]=='@'&&a[2*i+1]=='@'){
            isleaf = true;
        }

        if(isleaf){
            if(!first) cout<<" ";
            cout<<a[i];
            first = false;
            cnt++;
        }
    }
    cout<<endl;
    cout<<cnt<<endl;

    return 0;
}
```

中序遍历二叉树

问题描述

输入一整数序列，建立二叉排序树，然后中序遍历。

输入说明

输入第一行为整数的个数n，第二行是具体的n个整数。

输出说明

建立二叉排序树，然后输出中序遍历的结果。

输入样例

5

1 6 5 9 8

输出样例

1 5 6 8 9

```
#include<bits/stdc++.h>
using namespace std;
struct Node{
    int data;
    Node*left;
    Node*right;
    Node(int x):data(x),left(NULL),right(NULL){}
```

```
};

Node* insertBST(Node* root ,int x){
    if(root==NULL){
        return new Node(x);
    }

    if(x<root->data){
        root->left = insertBST(root->left,x);
    }else{
        root->right = insertBST(root->right,x);
    }

    return root;
}

void inorder(Node* root){
    if(!root) return;
    inorder(root->left);
    cout<<root->data<<" ";
    inorder(root->right);
}
int main(){
    int n;
    cin >> n;

    Node * root = NULL;
    for(int i=0;i<n;i++){
        int x;
        cin>>x;
        root = insertBST(root,x);
    }

    inorder(root);

    return 0;
}
```

二叉排序树的建立

```
Node* insertNode(Node* root,int x){
    if(root==nullptr){
        root = new Node{x,nullptr,nullptr};
        return root;
    }
    if(x<root->data){
        root->left = insertNode(root->left,x);
    }else if(x>root->data){
        root->right = insert(root->right,x);
    }
    return root;
}
```

二叉树遍历

给定一棵二叉树的后序遍历和中序遍历，请你输出其层序遍历的序列。这里假设键值都是互不相等的正整数。

输入说明

输入第一行给出一个正整数 N（<=100），表示某二叉树中结点的个数。第二行给出其后序遍历序列，第三行给出其中序遍历序列。数字间以一个空格分隔。

输出说明

在一行中输出该二叉树的层序遍历的序列。数字间以一个空格分隔。

样例输入

7

2 3 1 5 7 6 4

1 2 3 4 5 6 7

样例输出

4 1 6 3 5 7 2

```
#include<bits/stdc++.h>
using namespace std;

struct Node{
    int val;
    Node* left;
    Node* right;
    Node(int v):val(v),left(NULL),right(NULL){}
};

vector<int> post,in;

Node* build(int postL,int postR,int inL,int inR){
    if(postL>popstR) return;
    int rootVal = post[postR];
    Node *root = new Node(rootVal);

    int k = inL;
    while(in[k]!=rootVal) k++;

    int leftSize = k-inL;
    root->left = build(postL,postL+leftSize -1,inL,k-1);
    root->right = build(postL+leftSize,postR,k+1,inR);

    return root;
}
int main(){
    int N;
    cin>>N;

    post.resize(N);
    in.resize(N);

    for(int i =0 ;i< N;i++) cin >> post[i];
    for(int i =0 ;i< N;i++) cin >> in[i];

    Node* root = build(0,N-1,0,N-1);

    queue<Node*> q;
    q.push(root);

    bool first = true;
```

```
    while(!q.empty()){
        Node* cur = q.front();
        q.pop();

        if(!first) cout<<" ";
        cout<<cur->val;
        first = false;

        if(cur->left) q.push(cur->left);
        if(cur->right) q.push(cur->right);
    }

    return 0;

}
```

给定一棵二叉树的后序遍历和中序遍历，请你输出其层序遍历的序列。

```
#include<bits/stdc++.h>
using namespace std;

struct Node{
    int data;
    Node* left;
    Node* right;
    Node(int v):val(v),left(NULL),right(NULL){}
};
vector<int> pre,in;
Node* build(int preL,int preR,int inL,int inR){
    if(preL>preR) return;

    int rootVal = pre[preL];
    Node* root = new Node(rootVal);

    int k = inL;
    while(in[k]!=rootVal) k++;

    int leftSize = k-inL;

    //左子树
    root->left  = build(preL+1,preL+leftSize,inL,k-1);
    //右子树
    root->right = build(preL+leftSize+1,preR,k+1,inR);

    return root;
}
int main(){
    int N;
    cin>> N;

    pre.resize(N);
    in.resize(N);

    for(int i=0;i<N;i++) cin >>pre[i];
    for(int i=0;i<N;i++) cin >>in[i];

    Node* root = build(0,N-1,0,N-1);
```

```
    queue<Node*> q;
    q.push(root);

    bool first = true;
    while(!q.empty()){
        Node* cur = q.front();
        q.pop();

        if(!first) cout<<" ";
        cout<<cur->val;
        first = false;

        if(cur->left) q.push(cur->left);
        if(cur->right) q.push(cur->right);
    }

    return 0;
}
```

层次遍历

```
    queue<Node*> q;
    q.push(root);

    bool first = true;
    while(!q.empty()){
        Node* cur = q.front();
        q.pop();

        if(!first) cout<<" ";
        cout<<cur->val;
        first = false;

        if(cur->left) q.push(cur->left);
        if(cur->right) q.push(cur->right);
    }
```

二叉排序树的判定

输入格式

第一行两个数n,root，分别表示二叉树有n个结点，第root个结点是二叉树的根。接下来共n行，第i行三个数val_i、left_i、right_i，分别表示第i个结点的值val是val_i，左儿子left是第left_i个结点，右儿子right是第right_i个结点。

节点0表示空。

1<=n<=100000,保证是合法的二叉树

输出格式

输出"true"如果给定二叉树是二叉排序树，否则输出"false"

样例输入

5 1

5 2 3

1 0 0

3 4 5

4 0 0

6 0 0

样例输出

false

```
#include<bits/stdc++.h>
using namespace std;
struct Node{
    int val;
    int left;
    int right;
};

int n,root;
vector<Node> tree;
bool isBST(int u,long long minVal,long long maxVal){
    if(u==0) return true;
    if(tree[u].val<=minVal||tree[u].val>=maxVal) return false;

    return
isBST(tree[u].left,minVal,tree[u].val)&&isBST(tree[u].right,tree[u].val,maxVal);
}
int main(){
    cin >> n >> root;
    tree.resize(n+1);

    for(int i=1;i<=n;i++){
        cin >> tree[i].val >> tree[i].left >> tree[i].right;
    }

    if(isBST(root,LLONG_MIN,LLONG_MAX)){
        cout<<"true"<<endl;
    }else{
        cout<<"false"<<endl;
    }
    return 0;
}
```

循环队列的实现和测试

问题描述：
下面的程序实现一个循环队列（Queue），包括创建指定容量的空队列（createQueue）、判断队列是否为空（isEmpty）、入队（in）、出队（out）、输出队列所有元素（output）等操作，评判系统中已实现 createQueue、isEmpty、output 以及 main 函数。
请定义并提交实现入队操作的函数 in 和出队操作的函数 out

```c
#include <stdio.h>
#include<stdlib.h>
#define ERROR 0
#define OK 1
typedef int ElemType;
typedef struct {
    ElemType* data; //队列空间首地址
    int maxSize; //队列空间的大小
    int rear; //尾指针，队列不空时指示队尾元素之后的位置
    int length; //队列中的元素个数
}Queue;
int createQueue(Queue *q, int n); //创建容量为 n 的空队列
int isEmpty(Queue q); //判断队列是否为空，若为空则返回 1，否则返回 0
void output(Queue q); //输出队列中从队头至队尾的所有元素，不修改队列
int in(Queue *q, ElemType e); //元素 e 入队列，成功时返回 OK，失败则返回 ERROR
//（队满时入队失败，队列空间不扩充）
int out(Queue *q, ElemType *e); //非空时队头元素出队（通过参数 e 返回队头元素）
//若出队成功则返回 OK，否则返回 ERROR
int main(){ //评测系统中使用这里定义的 main 函数进行测试
    Queue Q;
    int n;
    scanf("%d",&n);
    if (createQueue(&Q,n)!=OK) return 0;
    ElemType elem;
    while (1) {
        scanf("%d", &elem);
        if (0==elem) break;
    if (in(&Q, elem)!=OK) {printf("OverFlow\n"); break;}
    }
    output(Q);
    int c = 4;
    while (c--) {
     if (out(&Q, &elem)) { printf("%d ", elem);}
     else { printf("UnderFlow\n"); break; }
    }
    return 0;
}//main
int createQueue(Queue *q, int n) {
     q->data = (ElemType *)malloc(n*sizeof(ElemType)); //申请队列空间
     if (!q->data) return ERROR;
     q->length = 0; q->rear = 0; q->maxSize = n;
     return OK;
}
int isEmpty(Queue q){
    return (0==q.length);
}
void output(Queue q) {
    if (isEmpty(q)) {
        return;
    }
    int idx = (q.rear - q.length + q.maxSize) % q.maxSize;
    do {
        printf("%d ", q.data[idx]);
        idx = (idx + 1) % q.maxSize;
    } while (idx != q.rear);
}
```

```
```

输入说明：
输入的第一个整数表示队列容量，之后是以 0 结尾的一个整数序列，整数之间以空格或换行符间隔。
输出说明：
按照 main 函数中安排的操作进行输出，参看下面的样例。
测试样例：
输入样例 1
8
1 2 3 4 5 0
输出样例 1
1 2 3 4 5 1 2 3 4
输入样例 1
3
1 2 3 4 5 0
输出样例 1
OverFlow
1 2 3 1 2 3 UnderFlow

```
int in(Queue *q, ElemType e) {
    if (q->length == q->maxSize) return ERROR; // 队满
    q->data[q->rear] = e;
    q->rear = (q->rear + 1) % q->maxSize;     // 循环
    q->length++;
    return OK;
}

int out(Queue *q, ElemType *e) {
    if (q->length == 0) return ERROR;         // 队空
    int front = (q->rear - q->length + q->maxSize) % q->maxSize;
    *e = q->data[front];
    q->length--;
    return OK;
}
```

二分查找

问题描述：
下面程序在指定的有序数组中用二分法（折半法，取下整）查找指定元素 key，评测系统中已实现 main 函数。请完成并提交实现二分查找运算的函数 biSearch（只需提交定义 biSearch函数的代码，该函数中需要输出与 key 进行比较的数组元素，每个元素后跟一个空格）

```
#include <stdio.h>
//在 n 个整数的有序数组 a 中查找 key，若成功则返回元素下标，否则返回-1
int biSearch(int n, int a[], int key);
int main() {//评测系统中使用这里定义的 main 函数进行测试
    int i, n, key;
    int a[101];
    scanf("%d%d",&n,&key);
    for(i=0;i<n;i++)
    scanf("%d", &a[i]);
    int position = biSearch(n, a, key);
```

```
    if (-1 == position) //查找失败
        printf("[Not found]\n");
    else
        printf("[Found at index %d]\n",position);
    return 0;
}
```

输入说明：
输入数据的第一行有两个整数，分别表示序列长度 n（1≤n≤100）和待查找的整数 key；从第二行开始，接下来的 n 个整数构成一个递增有序的序列。
输出说明：
在第一行上输出查找过程中参与比较的各整数，每个整数后跟一个空格，若查找成功，则接下来输出“[Found at index %d]”，其中%d 替换为元素的下标，否则输出“[Not found]”。
测试样例：
输入样例 1
6 18
4 11 15 18 23 35
输出样例 1
15 23 18 [Found at index 3]
输入样例 2
10 99
4 11 15 18 23 35 41 53 67 88
输出样例 2
23 53 67 88 [Not found]

```
int biSearch(int n,int a[],int key){
    int low = 0,high = n-1;

    while(low<=high){
        int mid = (low+high)/2;
        printf("%d ",a[mid]);

        if(a[mid]==key) return mid;
        else if(a[mid]<key) low = mid + 1;
        else high = mid - 1;
    }

    return -1;
}
```

报数出列问题

问题描述

已知N个人（以编号1，2，3，...，N分别表示）排成一列, 第一轮从编号为1的人开始依次报数，数到2的倍数的人出列；第二轮从头开始依次报数，数到3的倍数的人出列；第三轮再次从头开始依次报数，数到2的倍数的人出列；第四轮从头开始依次报数，数到3的倍数的人出列；依此规律重复下去，直到队列中的人数不超过三个为止。要求输出此时队列中剩下的人在初始队列中的编号。

问题输入

一个正整数N，表示初始人数，N≤5000。

问题输出

输出队列中剩下的人在初始队列中的编号，编号之间有一个空格。

输入样例

20

输出样例

1 7 19

提示

可以使用队列，每次操作时对队列中元素进行判断，不需要舍弃的就重新加入队列；

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int N;
    cin >> N;

    queue<int> people;
    //这里编号是为了最后输出在初始队列中的编号
    for(int i=1;i<=N;i++){
        people.push(i);
    }

    int round = 1;

    while(people.size()>3){
        int k = (round % 2 == 1) ? 2 : 3;//奇数轮是2倍数，偶数轮是3倍数
        int count = 1;
        int size = people.size();

        //遍历当前队列
        for(int i=0;i<size;i++){
            int person = people.front();
            people.pop();

            //这里不能用person%k,因为在之后几轮中person表示的数字，不是报号的数字，因为会
有人出队列
            if(count%k!=0){
                people.push(person);
            }
            count++;
        }
        round ++;
    }

    while(!people.empty()){
        cout<<people.front();
        people.pop();
        if(!people.empty()) cout<<" ";
    }
    cout<<endl;
    return 0;
}
```

奇偶序列

问题描述
输入一个整数组成的序列，然后将序列中的奇数位置结点依序放在前面，偶数位置结点依序放在后面，组成一个新的序列。输出此新序列。
问题输入
输入共两行，第一行是一个正整数n，表示序列长度，n<100。第二行是n个整数，中间用空格隔开，表示序列中的元素。
问题输出
在一行上输出新生成的序列，数字中间用空格隔开。
输入样例
7
10 2 30 4 50 60 7
输出样例
10 30 50 7 2 4 60

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n;
    cin >> n;
    vector<int> nums(n);
    for(int i=0;i<n;i++){
        cin >> nums[i];
    }

    vector<int> result;//这里不能用vector<int> result(n); 这样相当于已经包含了n个0元素

    //先放奇数位置
    for(int i=0;i<n;i+=2){
        result.push_back(nums[i]);
    }
    //再放偶数位置
    for(int i =1;i<n;i+=2){
        result.push_back(nums[i]);
    }
    //输出结果
    for(int i=0;i<n;i++){
        cout<<result[i];
        if(i!=n-1) cout<<" ";
    }
    cout<<endl;
    return 0;
}
```

判断堆栈出栈序列是否有效

描述：
如果以序列“1,2,3,4”作为一个栈（初始为空）的输入，那么可得到输出序列“1,2,3,4”或“4,3,2,1”或“2,3,1,4”等等，但是肯定得不到输出序列“4,1,2,3”或“3,1,2,4”等等。请编写一个程序，判断能否通过一个栈得到给定的输出序列。
输入说明：
每组数据由两行构成，第一行只有一个整数n（<10），表示序列长度，同时表示栈的输入序列为“1,2,3,...,n”；第二行为n个整数，表示需要判断的出栈序列，整数之间以空格分隔。
输出说明：
输出一个yes或no（表示能否通过栈得到该序列）。

输入样例:
6
3 4 2 1 5 6
输出样例:
yes
提示
根据栈的后进先出特性进行判断

思路：模拟法

1. 使用一个栈来模拟操作
2. 使用一个指针 `current` 表示下一个要压入栈的数字（从1开始到n）
3. 遍历给定的输出序列：
    - 如果栈顶元素等于当前要输出的元素 → pop
    - 否则，将 `current` 及之后的数字依次压入栈，直到栈顶等于当前要输出的元素
    - 如果 `current` 已经超过n且栈顶不等于要输出的元素 → 不可能序列

```
#include<bits/stdc++.h>
using namespace std;
bool isValidSequence(int n,vector<int>& seq){
    stack<int> st;
    int current = 1;//下一个要压入栈的数字
    for(int i=0;i<n;i++){
        int target = seq[i];//当前需要输出的数字

        //如果栈顶不是目标数字，就压入数字直到栈顶是目标数字
        while(st.empty()||st.top()!=target){
            if(current>n){
                return false;//已经没有数字可压入；
            }

            st.push(current);
            current++;
        }
        //此时栈顶一定是目标数字
        st.pop();
    }
    return true;
}
int main(){
    int n;
    while(cin>>n){//支持多组数据
        vector<int> seq(n);
        for(int i=0;i<n;i++){
            cin >>seq[i];
        }

        if(isValidSequence(n,seq)){
            cout<<"yes"<<endl;
        }else{
            cout<<"no"<<endl;
        }
    }
```

```
    return 0;
}
```

问题描述

给定一个栈，其中最多存储M个数据。将N个数据以1,2,3,...,N的顺序压栈，然后再随机弹栈。判断一下哪些是有可能的弹栈顺序，而哪些不是。例如M是5，N是7，我们可以得到1, 2, 3, 4, 5, 6, 7的弹栈顺序，而不能得到3, 2, 1, 7, 5, 6, 4这样的弹栈顺序。（M，N<=1000）

输入说明

输入包含了一种情况下的测试数据。在每种情况下，有三组输入数据：

第一组列在首行，包含了三个数据：栈的最大容量M，压栈数据长度N，要验证的弹栈序列个数K。

第二组为压栈数据：N个整数s1, s2, ..., sn (1 ≤ si ≤ 10000, 1 ≤ i ≤ n)。

第三组为K行数据，每行包括了一个需要验证的弹栈序列（含N个数据）。

同行数据间以空格隔开。

输出说明

输出应该包括K行，包行了对每个输入的验证序列，输出的一个判断结果。如果这个弹出序列可能发生，就输出“YES”，否则就输出“NO”。

输入样例

5 7 5

1 2 3 4 5 6 7

1 2 3 4 5 6 7

3 2 1 7 5 6 4

7 6 5 4 3 2 1

5 6 4 3 7 2 1

1 7 6 5 4 3 2

输出样例

YES

NO

NO

YES

NO

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int M,N,K;
    cin >> M >> N >> K;
    vector<int> inputSeq(N);
    for(int i =0 ;i< N ;i++) cin >> inputSeq[i];

    for(int t =0;t<K;t++){
        vector<int> outSeq(N);
```

```
        for(int i=0;i<N;i++) cin >> outSeq[i];
        stack<int> st;
        int j=0;
        bool flag = true;
        for(int i=0;i<N;i++){
            st.push(inputSeq[i]);

            //入栈先检查容量 栈会自动扩容
            if((int)st.size()>M){
                flag = false;
                break;
            }

            while(!st.empty()&&j<N&&st.top()==outSeq[j]){
                st.pop();
                j++;
            }
        }

        if(flag&&st.empty()){ cout<<"YES"<<endl;}else{cout<<"NO"<<endl;}
    }
    return 0;
}
```

二叉树的不同形态

问题描述
给定二叉树T（树深度H<=10，深度从1开始，结点个数N<1024，结点编号1~N）的层次遍历序列和中序遍历序列，输出T从左向右叶子结点以及二叉树先序和后序遍历序列。
输入格式
输入共三行：第一行是整数n，表示二叉树中的结点数目；第二行有n个整数，表示该二叉树的层次遍历序列；第三行也是n个整数，表示该二叉树的中序遍历序列。整数间以空格隔开。
输出格式
输出三行，分别是：从左向右的叶子结点，先序遍历序列，后序遍历序列。结点编号用空格隔开。
样例输入
7
3 5 4 2 6 7 1
2 5 3 6 4 7 1
样例输出
2 6 1
3 5 2 4 6 7 1
2 5 6 1 7 4 3

1. 重建二叉树
   层次遍历的第一个节点是根节点
   在中序遍历中找到根节点，左边是左子树，右边是右子树
   递归构建左右子树
2. 关键点
   层次遍历不能直接确定子树结构，需要结合中序遍历
   对于每个子树，从层次遍历中筛选出属于该子树的节点，并保持层次遍历的顺序
3. 算法步骤
   从层次遍历中取第一个节点作为当前子树的根

在中序遍历中找到根节点的位置，划分左右子树
从层次遍历中筛选出属于左子树的节点（保持原顺序）
从层次遍历中筛选出属于右子树的节点（保持原顺序）
递归构建左右子树

```
#include<bits/stdc++.h>
using namespace std;
struct Node{
    int val;
    Node* left;
    Node* right;
    Node(int x):val(x),left(NULL),right(NULL){}
};

Node* build(vector<int>&lev,vector<int>&in){
    if(lev.empty()||in.empty()) return NULL;

    int rootVal = lev[0];
    Node * root = new Node(rootVal);

    //在中序中找根位置
    int pos = 0;
    while(in[pos]!=rootVal) pos++;

    //分割中序
    vector<int> leftIn(in.begin(),in.begin()+pos);
    vector<int> rightIn(in.begin()+pos+1,in.end());

    //从层次遍历中选出左右子树的结点
    vector<int> leftLev,rightLev;
    unordered_set<int> leftSet(leftIn.begin(),leftIn.end());
    //跳过根结点
    for(int i=1;i<lev.size();i++){
        if(leftSet.count(lev[i])){
            leftLev.push_back(lev[i]);
        }else{
            rightLev.push_back(lev[i]);
        }
    }

    //递归建树
    root->left = build(leftLev,leftIn);
    root->right = build(rightLev,rightIn);

    return root;
}
//输出叶子结点 注意是：从左向右的叶子结点
void printLeaves(Node*root,bool&first){
    if(!root) return;

    if(!root->left&&!root->right){
        if(!first) cout<<" ";
        cout<<root->val;
        first = false;
        return;
    }
    printLeaves(root->left,first);
```

```
    printLeaves(root->right,first);
}
void printPreorder(Node* root,bool &first){
    if(!root) return;

    if(!first) cout<<" ";
    cout<<root->val;
    first = false;

    printPreorder(root->left,first);
    printPreorder(root->right,first);
}

void printPostorder(Node* root,bool &first){
    if(!root) return;

    printPostorder(root->left,first);
    printPostorder(root->right,first);
    if(!first) cout<<" ";
    cout<<root->val;
    first = false;
}
int main(){
    int n;
    cin>> n;

    vector<int> lev(n),in(n);
    for(int i=0;i<n;i++) cin>>lev[i];
    for(int i=0;i<n;i++) cin>>in[i];

    Node* root = build(lev,in);

     // 输出叶子节点
    bool first = true;
    printLeaves(root, first);
    cout << endl;

    // 输出先序遍历
    first = true;
    printPreorder(root, first);
    cout << endl;

    // 输出后序遍历
    first = true;
    printPostorder(root, first);
    cout << endl;

    return 0;
}
```

是否为堆

试题名称　　是否为堆

问题描述：

对于 n 个元素的关键字序列$\{K_1, K_2, \cdots, K_n\}$，当且仅当满足下列关系时称其为堆，其中 $2i$ 和 $2i+1$ 应不大于 n。

$$\begin{cases} K_i \le K_{2i} \\ K_i \le K_{2i+1} \end{cases} \quad 或 \quad \begin{cases} K_i \ge K_{2i} \\ K_i \ge K_{2i+1} \end{cases}$$

现给出 n 个关键子构成的序列，请编程判断其是否为堆。

输入说明：

输入为两行，第一行为一个正整数 n（2≤n≤100），表示关键字序列的长度。第二行为 n 个整数，表示关键字序列的元素。

输出说明：

如果输入的整数序列是小顶堆，则输出“Min heap”，若是大顶堆，则输出“Max heap”，否则输出“Not heap”，换行。

测试样例：

输入样例 1
6
12 36 24 85 47 30

输出样例 1
Min heap

输入样例 2
7
91 36 53 12 30 85 24

输出样例 2
Not heap

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n;
    cin >> n;

    vector<int> arr(n);
    for(int i=0;i<n;i++){
        cin >> arr[i];
    }

    bool isMaxHeap = true;//假设是大顶堆
    bool isMinHeap = true;//假设是小顶堆

    //只遍历非叶子结点
    for(int i = 0;i<n/2 ;i++){
        int left  = 2*i + 1;
```

```
        int right = 2*i + 2;//下标从0开始

        //检查左孩子
        if(left<n){
            if(arr[i]<arr[left]){
                isMaxHeap = false;
            }
            if(arr[i]>arr[left]){
                isMinHeap = false;
            }
        }

        //检查右孩子
        if(right<n){
            if(arr[i]<arr[right]){
                isMaxHeap = false;
            }
            if(arr[i]>arr[right]){
                isMinHeap = false;
            }
        }

        if(!isMaxHeap&&!isMinHeap){
            break;//提前结束
        }
    }

    if(isMaxHeap){
        cout<<"Max heap"<<endl;
    }else if(isMinHeap){
        cout<<"Min heap"<<endl;
    }else{
        cout<<"Not heap"<<endl;
    }

    return 0;

}
```

计算AOE网中的关键路径的长度

输入格式

输入数据第一行是一个正整数，表示图中的顶点个数n（顶点将分别按0，1，…，n-1进行编号），顶点数不超过100，其中0为源点，n-1为汇点。之后的n行每行都包含n个整数，为AOE-网的邻接矩阵，其中0表示两个顶点间无直接可达的弧，大于0的整数表示活动持续的时间。

输出格式

输出AOE-网中关键路径的长度，如果网中有环，则输出“NO”。

样例输入

9

0 6 4 5 0 0 0 0 0

0 0 0 0 1 0 0 0 0

0 0 0 0 1 0 0 0 0

0 0 0 0 0 2 0 0 0

0 0 0 0 0 0 9 7 0

0 0 0 0 0 0 0 4 0

0 0 0 0 0 0 0 0 2

0 0 0 0 0 0 0 0 4

0 0 0 0 0 0 0 0 0

样例输出

18

输出说明：如果网中有环，则示例输出如下：

NO

- 关键路径长度是整个工程的最短完成时间，也就是从源点到汇点的最长路径的长度
- 最早开始时间：一个事件（顶点）最早可以发生的时间
- 最晚开始时间：一个事件最晚必须发生的时间，否则会影响工期
- 关键路径：所有最早开始时间=最晚开始时间的顶点构成的路径
- 关键路径长度=汇点的最早开始时间=汇点的最晚开始时间

> 只要求出每个点的最早开始时间，得出汇点的最早开始时间，就是关键路径长度

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n;
    cin >> n;

    vector<vector<int>> g(n,vector<int>(n));
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            cin >> g[i][j];
        }
    }

    //1.拓扑序列检测环
    vector<int> indegree(n,0);//用来存储每个顶点的入度
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(g[i][j]>0) indegree[j]++;
        }
    }

    //创建一个队列q,用于广度优先遍历，把入度为0的顶点加入队列，入度为0表示没有边指向它，可以作为起点
    queue<int> q;
    for(int i=0;i<n;i++){
        if(indegree[i]==0) q.push(i);
    }

    //创建向量topo，用于存储拓扑排序的结果
    //当队列不为空时循环
```

```
    vector<int> topo;
    while(!q.empty()){
        //从队列中取出队首顶点u,将u加入拓扑序列
        int u = q.front();q.pop();
        topo.push_back(u);

        //遍历顶点u所有的出边，如果存在u->v,将顶点v的入度减1，如果v的顶点变为0，把v加入队列
        for(int v =0 ;v < n;v++){
            if(g[u][v]>0&&--indegree[v]==0){
                q.push(v);
            }
        }
    }

    //如果拓扑序列的长度不等于顶点数，说明图中有环
    if(topo.size()!=n){
        cout<<"NO"<<endl;
        return 0;
    }

    //2.计算最早开始时间
    vector<int> earliest(n,0);
    //按拓扑序列遍历每个顶点u,对于u的每个后继顶点v,更新最早开始时间
    for(int u:topo){
        for(int v = 0;v<n;v++){
            if(g[u][v]>0){
                earliest[v] = max(earliest[v],earliest[u]+g[u][v]);
            }
        }
    }

    //3.输出关键路径长度（汇点最早时间）
    cout<<earliest[n-1]<<endl;

    return 0;
}
```

*图的广度优先

问题描述
已知无向图的邻接矩阵，以该矩阵为基础，给出广度优先搜索遍历序列，并且给出该无向图的连通分量的个数。在遍历时，当有多个点可选时，优先选择编号小的顶点。（即从顶点1开始进行遍历）
输入格式
　第一行是1个正整数，为顶点个数n（n<100），顶点编号依次为1，2，…，n。后面是邻接矩阵，n行n列。
输出格式
共2行。第一行输出为无向图的广度优先搜索遍历序列，输出为顶点编号，顶点编号之间用空格隔开；第二行为无向图的连通分量的个数。
样例输入
　6
0 1 0 0 0 0
1 0 0 0 1 0
0 0 0 1 0 0
0 0 1 0 0 0
0 1 0 0 0 1

0 0 0 0 1 0
样例输出
1 2 5 6 3 4
2

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    // 读取邻接矩阵
    vector<vector<int>> graph(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> graph[i][j];
        }
    }

    vector<bool> visited(n, false);  // 访问标记
    vector<int> bfsOrder;            // BFS遍历序列
    int componentCount = 0;          // 连通分量个数

    // 遍历所有顶点
    for (int start = 0; start < n; start++) {
        // 如果顶点未访问，开始一次新的BFS（新的连通分量）
        if (!visited[start]) {
            componentCount++;  // 连通分量+1

            // BFS遍历当前连通分量
            queue<int> q;
            q.push(start);
            visited[start] = true;

            while (!q.empty()) {
                int u = q.front();
                q.pop();
                bfsOrder.push_back(u + 1);  // 转换为1-based编号

                // 将当前顶点的所有邻接点按编号从小到大加入队列
                for (int v = 0; v < n; v++) {
                    if (graph[u][v] && !visited[v]) {
                        q.push(v);
                        visited[v] = true;
                    }
                }
            }
        }
    }

    // 输出BFS遍历序列
    for (int i = 0; i < bfsOrder.size(); i++) {
        cout << bfsOrder[i] << " \n"[i == bfsOrder.size() - 1];
    }

    // 输出连通分量个数
```

```
    cout << componentCount << endl;

    return 0;
}
```

*图的深度优先遍历

问题描述
已知无向图的邻接矩阵，以该矩阵为基础，给出深度优先搜索遍历序列，并且给出该无向图的连通分量的个数。在遍历时，当有多个点可选时，优先选择编号小的顶点。（即从顶点1开始进行遍历）
输入格式
　第一行是1个正整数，为顶点个数n（n<100），顶点编号依次为1，2，…，n。后面是邻接矩阵，n行n列。
输出格式
共2行。第一行输出为无向图的深度优先搜索遍历序列，输出为顶点编号，顶点编号之间用空格隔开；第二行为无向图的连通分量的个数。
样例输入
　6
0 1 0 0 0 0
1 0 0 0 1 0
0 0 0 1 0 0
0 0 1 0 0 0
0 1 0 0 0 1
0 0 0 0 1 0
样例输出
　1 2 5 6 3 4
2

```
#include <bits/stdc++.h>
using namespace std;

void dfs(int u, vector<vector<int>>& graph, vector<bool>& visited, vector<int>&
dfsOrder) {
    visited[u] = true;
    dfsOrder.push_back(u + 1);  // 转换为1-based编号

    // 优先选择编号小的顶点
    for (int v = 0; v < graph.size(); v++) {
        if (graph[u][v] && !visited[v]) {
            dfs(v, graph, visited, dfsOrder);
        }
    }
}

int main() {
    int n;
    cin >> n;

    // 读取邻接矩阵
    vector<vector<int>> graph(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> graph[i][j];
        }
```

```
    }

    vector<bool> visited(n, false);
    vector<int> dfsOrder;
    int componentCount = 0;

    // 从顶点1开始（下标0） 只是为了表明从小到大，0和其余的合并在一起也可以
    if (!visited[0]) {
        componentCount++;
        dfs(0, graph, visited, dfsOrder);
    }

    // 继续遍历其他未访问的顶点（其他连通分量）
    for (int i = 1; i < n; i++) {
        if (!visited[i]) {
            componentCount++;
            dfs(i, graph, visited, dfsOrder);
        }
    }

    // 输出DFS遍历序列
    bool first = true;
    for (int i = 0; i < dfsOrder.size(); i++) {
        if(!first) cout<<" ";
        cout << dfsOrder[i];
        first = false;
    }
    cout<<endl;

    // 输出连通分量个数
    cout << componentCount << endl;

    return 0;
}
```

最短路径

问题描述
求图中任意两个顶点之间的最短路径。
输入格式
输入数据第一行是一个正整数，表示图中的顶点个数n（顶点将分别按0，1，…，n-1进行编号）。之后的n行每行都包含n个整数，第i行第j个数表示顶点i-1和顶点j-1之间的边长，用10000来表示两个顶点之间无边。后面每行2个数字，表示一对待求最短路径的顶点，用-1 -1表示输入结束，-1 -1不求解。
输出格式
每对待求最短路径的顶点输出两行数据：第一行输出两个顶点间的最短路径长度，第二行输出最短路径，要按顺序输出顶点编号序列，顶点间用空格隔开。当两个顶点间没有路径时，只在一行上输出字符串“NO”。
样例输入
7
0 12 10000 10000 10000 10000 10000
12 0 10000 10000 3 10000 10000
10000 10000 0 10000 10000 21 11
10000 10000 10000 0 10000 10000 10000
10000 3 10000 10000 0 10000 8
10000 10000 21 10000 10000 0 10000

10000 10000 11 10000 8 10000 0
0 2
0 3
5 0
2 1
1 5
-1 -1
样例输出
34
0 1 4 6 2
NO
55
5 2 6 4 1 0
22
2 6 4 1
43
1 4 6 2 5

- `next[i][j]`记录的是:从i到j的最短路径上，i之后的下一个顶点
- 初始时，如果有直接边(i,j)，那么`next[i][j]=j`直接走到
- 否则`next[i][j]=-1`表示没有直接路径
- 外层循环，中间顶点k,逐步考虑能否通过顶点k来缩短i到j的路径
- k从0到n-1,考虑所有顶点作为中间点的可能性
- 内层双重循环：遍历所有顶点对(i,j)
- 条件检查：
  1. `dist[i][k] < INF`：i到k有路径
  2. `dist[k][j] < INF`：k到j有路径
  3. `dist[i][k] + dist[k][j] < dist[i][j]`：通过k的路径比当前路径更短
- 核心更新：找到更短路径，更新距离
- 从i到j的新最短距离 = i到k的距离 + k到j的距离

最小周期串

描述：
如果一个字符串可以由某个长度为k的字符串重复多次得到，我们说该串以k为周期。例如，abcabcabcabc以3为周期（注意，它也以6和12为周期）。输入一个长度不超过80的串，输出它的最小周期。
输入说明：
每组数据为一个字符串。
输出说明：
输出每组字符串的最小周期。
输入样例：
GeeGeeGee
输出样例：
3

```
#include<iostream>
#include<string>
#include<vector>
using namespace std;
```

```
vector<int> computeLPS(const string& s){
    int n = s.size();
    vector<int> pi(n,0);
    int j=0;
    for(int i=1;i<n;i++){
        while(j>0&&s[i]!=s[j]) j = pi[j-1];//不匹配时回退
        if(s[i]==s[j]) ++j;//匹配时长度加1
        pi[i] = j;
    }

    return pi;
}
int main(){
    string  s;
    if(!(cin>>s)) return 0;
    int n=s.size();
    vector<int> pi = computeLPS(s);


    int l = pi[n-1];//整个字符串的最长相等前后缀的长度
    int p = n-l;//可能的循环节长度
    if(n%p==0) cout<<p<< endl;
    else cout<<n<<endl;

    return 0;
}
```

迷宫问题

```
#include<bits/stdc++.h>
using namespace std;
vector<vector<int>> maze;
vector<vector<bool>> visited;
vector<tuple<int,int,int>> path;


bool found = false;

int dx[4] = {0,1,0,-1};
int dy[4] = {1,0,-1,0};

int dir[4] = {1,2,3,4};

void dfs(int x,int y){
    if(found) return;
    if(x==ey-1&&y==ey-1){
        found = true;
        path.push_back({ex,ey,1});
        return;
    }
    visited[x][y] = true;

    for(int k=0;k<4;k++){
        int nx = x + dx[k];
        int ny = y + dy[k];
```

```
        if(nx>=0&&nx<m&&ny>=0&&ny<n&&maze[nx][ny]==0&&visited[nx][ny]==false){
            path.push_back({x+1,y+1,dir[k]});
            dfs(nx,ny);
            if(found) return;

            path.pop_back();
        }
    }
}
int main(){
    cin >> m >> n;
    cin >> sx >> sy >>ex >>ey;
    maze.assign(m,vector<int>(n,0));
    visited.assign(m,vector<bool>(n,false));

    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            cin >> maze[i][j];
        }
    }

    if(sx==ex&&sy==ey){
        cout<<"("<<sx<<","<<sy<<","<<"1"<<")"<<endl;
        return 0;
    }
    path.clear();
    dfs(sx-1,sy-1);

    if(!found){
        cout<<"no"<<endl;
    }else{
        for(int i=0;i<path.size();i++){
            auto t = path[i];
            cout<<"("<<get<0>(t)<<","<<get<1>(t)<<","<<get<2>(t)<<")";
            if(i!=path.size()-1) cout<<",";
        }
        cout<<endl;
    }
    return 0;
}
```

括号匹配-（用栈实现）

```
#include<bits/stdc++.h>
using namespace std;
bool isMatch(const string &expr){
    stack<char> s;
    for(char c:expr){
        if(c=='('||c=='{'||c=='['){
            s.push(c);
        }else if(c=='}'||c==')'||c==']'){
            if(s.empty) return false;
            char topChar = s.top();
            if(c=='}'&&topChar=='{'||c==')'&&topChar=='('||c==']'&&topChar==']')
{
                s.pop();
```

```
            }else{
                return false;
            }
        }
    }
    return s.empty();
}
int main(){
    int n;
    cin >> n;
    for(int i=0;i<n;i++){
        string expr;
        cin >>expr;

        if(isMatch(expr)){
            cout<<"yes"<<endl;
        }else{
            cout<<"no"<<endl;
        }
    }
    return 0;
}
```

后缀表达式求值

- 从左往右入栈，先入栈的先算

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    string str;
    cin>> str;
    int n = str.length();
    stack<int> st;
    for(int i=0;i<n;i++){
        if(isdigit(str[i])){
            st.push(str[i]-'0');
        }else{
            int b = st.top();st.pop();
            int a = st.top();st.pop();

            int result = 0;
            if(str[i]=='+') result = a + b;
            else if(str[i]=='-') result = a - b;
            else if(str[i]=='*') result = a * b;
            else if(str[i]=='/') result = a/b;
            st.push(result);
        }
    }

    cout<<st.top()<<endl;
    return 0;
}
```

前缀表达式求值

- 从后往前入栈，后入栈的先计算

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    /*
        reverse是原地修改，没有返回值
        错误：string str = reverse(s);
    */
    //正确方法如下
    string str;
    cin >>str;
    reverse(str.begin(),str.end());

    int n = str.length();
    stack<int> st;
    for(int i=0;i<n;i++){
        if(isdigit(str[i])){
            st.push(str[i]-'0');
        }else{
            int a = st.top();st.pop();
            int b = st.top();st.pop();

            int result=0;
            if(str[i]=='+') result = a + b;
            else if(str[i]=='-') result = a - b;
            else if(str[i]=='*') result = a*b;
            else if(str[i]=='/') result = a/b;
            st.push(result);
        }
    }

    cout<<st.top()<<endl;

    return 0;

}
```

模式匹配

题目描述

串的匹配问题。给定两个字符串S="s1s2...sn" 和T="t1t2...tm"，在主串S（又称为目标串）中查找模式串T的过程称为串匹配，也叫做模式匹配。如果T是S的子串，返回T在S中首次出现的位置（默认字符串第一个字符的下标为1），如若不是则返回0。

输入格式:

输入为两行，第一行为主串S，第二行为模式串T。

输出格式:

输出T在S中首次出现的位置。如果T不是S的子串，则输出0.

输入样例:

ABABABC

ABA

输出样例:

1

模式匹配BF算法

```
#include<bits/stdc++.h>
using namespce std;
int main(){
    string S,T;
    cin >>S>>T;

    //length size可以互换使用完全一样
    int n = S.length();
    iny m = T.length();
    int pos=0;

    for(int i=0;i<n;i++){
        int j=0;
        while(j<m&&S[i+j]==T[j]) j++;
        if(j==m){
            pos = i + 1;
            break;
        }
    }

    cout<< pos<<endl;

}
```

[模式]串求next数组

```
vector<int> computeNext(const string &T){
    int m = T.size();
    vector<int> next(m,0);

    int j=0;

    for(int i=1;i<m;i++){
        while(j>0&&T[i]!=T[j]){
            j = next[j-1];
        }
        if(T[i]==T[j]){
            j++;
        }
        next[i]=j;
    }

    return next;
}
```

中心对称的字符串

- 用reverse翻转来完成

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n;
    string s;

    cin>>n;
    cin>>s;

    string left = s.substr(0,n/2);//左边最后一个字符位置是 n/2 -1
    int right_start ;
    if(n%2==0)right_start = n/2;
    else right_start = n/2+1;

    string right = s.substr(right_start,n/2);

    //翻转后半部分
    reverse(right.begin(),right.end());

    cout<<(left == right ? "YES":"NO")<<endl;

    return 0;
}
```

约瑟夫环

```
#include<bits/stdc++.h>
using namespace std;
struct Node{
    int id;
    int key;
    Node* next;
};
Node* createList(int n){
    Node* head = nullptr;
    Node* p = nullptr;//尾指针

    for(int i=1;i<=n;i++){
        Node* node = new Node;
        node->id = i;
        cin >> node->key;

        if(head==nullptr){
            head = node;
            node->next = head;
        }else{
            node->next = head;
            p->next = node;
        }

        p = node;
    }

    return head;
```

```
}
void josephus(Node* head,int m){
    Node* p = head;
    Node *pre = nullptr;

    while(p->next!=p){
        for(int i=1;i<m;i++){
            pre = p;
            p = p->next;
        }

        cout<<p->id<<" ";

        m = p->key;

        pre->next = p->next;
        p = p->next;
        Node * temp= p;
        delete temp;
    }

    cout<<p->id<<endl;

    delete p;
}
int main(){
    int n,m;
    cin >> n >> m;
    Node* head = createList(n);
    josephus(head,m);

    return 0;
}
```

输出完全二叉树的某一层

问题描述

对一棵完全二叉树，输出某一深度的所有节点，有则输出这些节点，无则输出EMPTY。

输入格式

输入有多组数据。

每组数据第一行输入一个结点数n(1<=n<=1000)，第二行将树中的这n个节点依次输入（每个结点存储的数据是一个数字），n个结点编号方式是层间从上到下、层内从左到右依次编号；第三行输入一个d代表深度。

当n=0时，表示输入结束。

输出格式

每组数据在一行上输出该树中第d层的所有节点，节点间用空格隔开。每组数据输出完成后要换行。

样例输入

4

1 2 3 4

2

0

样例输出

2 3

样例说明

该完全二叉树的第一层是1，第二层是2 3，第三层是4；题目要求输出第二层，则输出2 3。

- 第i层的第一个节点 2^(i-1)
- 第i层的最后一个节点 2^i-1

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n;
    while(cin>>&&n!=0){
       vector<int> arr(n+1);
       for(int i=1;i<n+1;i++){
           cin >> arr[i];
       }

        int d;
        cin >> d;

        int start = pow(2,d-1);
        int end = pow(2,d)-1;

        if(start>n){
            cout<<"EMPTY"<<endl;
            continue;
        }

        //重要的是不要忘记 i<=n 这个关键条件
        for(int i=start;i<=end&&i<=n;i++){
            if(i!=start) cout<<" ";
            cout<<arr[i];
        }
        cout<<endl;
    }
    return 0;
}
```

最近公共父节点

问题描述

有一棵无限大的完全二叉树，该二叉树自上而下、自左而右从1开始编号。从某一个结点到根结点（编号是1的结点）都有一条唯一的路径，比如从5到根结点的路径是（5, 2, 1），从4到根结点的路径是（4, 2, 1），从根结点1到根结点的路径上只包含一个结点1，因此路径就是（1）。对于两个结点x和y，假设他们到根结点的路径分别是（x1, x2, ... ,1）和（y1, y2,...,1），那么必然存在两个正整数i和j，使得从xi 和yj 开始，有xi = yj，xi + 1 = yj + 1，xi + 2 = yj + 2，...

现在的问题就是，给定x和y，要求他们的最近公共父节点，即xi（也就是 yj）。

输入格式

输入包含多组数据，每组数据包含两个正整数x和y（1≤x, y≤2^31-1），x=0、y=0表示输入结束。

输出格式

对应每一组数据，输出一个正整数xi，即它们的首个公共父节点。每输出一个数字后要换行。

样例输入

10 4

0 0

样例输出

2

样例说明

结点10到根结点的路径为（10,5,2,1），结点4到根节点的路径为（4,2,1），所以他们的首个公共父结点为2。

- 这段代码的核心原理是：在完全二叉树中，不断将较大的节点编号除以2向上回溯父节点，直到两个节点相遇

```
#include<bits/c++.h>
using namespace std;
int main(){
    long x,y;
    while(cin >> x >> y){
        if(x==0&&y==0) break;

        while(x!=y){
            if(x>y) x/=2;
            else y/=2
        }

        cout<<x<<endl;
    }
    return 0;
}
```

直接插入排序

```
#include<bits/c++.h>
using namespace std;
int main(){
    int n;
    cin >> n;
    vector<int> arr(n);
    for(int i=0;i<n;i++){
        cin >> arr[i];
    }

    for(int i=1;i<n;i++){
        int temp = arr[i];
        int j = i -1;
        while(j>=0&&arr[j]>temp){
```

```
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;

        for(int k=0;k<=i;k++){
            cout<<arr[k]<<" ";
        }
        cout<<endl;
    }

    return 0;
}
```